Sprint Review and Retrospective

Nathan Johnson

Southern New Hampshire University

CS 250: Software Development Lifecycle

Haruka Konishi

October 18, 2026

Sprint Review and Retrospective

As Scrum Master reviewing the SNHU Travel project, I would focus on what the completed work showed and what needs improvement. Taking on different roles throughout the course helped me see how requirements, development, testing, and communication fit together. The project was a simulation, and my completed work included a team charter, user stories, test cases, and a customized Java slideshow. Each assignment gave me another way to look at the same customer's needs.

The different roles contributed to that work in specific ways. As Product Owner, I prioritized and sized Personalized Destination Recommendations, Maximum Price Filter, and Vacation Type Selection. As a developer, I customized the destination display with pictures, descriptions, and design changes. As a tester, I created three initial test cases and three revised cases for the updated interface. My team charter established communication guidelines and directed questions about priorities to the Product Owner and obstacles to the Scrum Master. From the Scrum Master’s perspective, these responsibilities connect the customer’s priorities with work that developers and testers can understand.

The Scrum-Agile approach helped connect user stories to clear completion requirements. My professor pointed out that the acceptance criteria were missing from my original story details. That feedback showed me that a clear value statement still leaves room for misunderstanding. My later Maximum Price Filter test case specified a $1,500 limit, packages below, at, and above that amount, and a clear message when no packages matched. It also checked that clearing the limit restored the available results. These were planned tests, rather

than proof that the filter had been implemented, but they made the expected behavior much clearer.

For the destination display, I worked through requirements, design, coding, and testing in smaller steps. I added destination information and images, adjusted the appearance, and checked the navigation. Changing the header to white with black lettering made the text easier to read. Checking the buttons and displayed information after changes helped me judge whether the updated display worked as intended. This connected the user story to something I could demonstrate and inspect instead of treating the code alone as evidence of completion.

The move from a destination list to a slideshow was a clear example of changing direction. My original tests involved scrolling through a full results list. The revised tests checked one destination at a time and used Next and Previous to verify navigation. Later, the wellness focus required changes to the pictures and descriptions. I could build on the existing program while updating the affected content and tests. In a team Sprint, I would have the Product Owner clarify the change and work with the Developers to assess its effect on the Sprint Goal before adjusting the plan.

My developer journal included communication samples that made the work more specific. I asked the Product Owner for “the final list and order of the five destinations” and clarification about image-credit placement. Those questions would help prevent developers from choosing content or a design that the customer had not approved. I also wrote to the tester, “If you find a problem, please include what steps caused it, what you expected to happen, and what happened.” That request gives the developer enough information to reproduce a problem. Both samples encourage collaboration because they ask for information the other role can provide and explain what is needed to move forward.

Excel helped organize the backlog and test cases because priorities, story sizes, steps, and expected results were easy to compare. Keeping the original and revised tests on separate sheets also made the interface changes visible. Its limitation was that updates depended on maintaining the spreadsheet carefully. Eclipse supported the development work by showing code problems and allowing me to run the application. Together, these tools helped me connect planned behavior with the program, although neither tool could resolve an unclear requirement without communication.

A shared task board would make that coordination easier for a working Scrum team. During Sprint Planning, the backlog and acceptance criteria would help the Developers select work they understood and could complete. During the Daily Scrum, they could inspect progress toward the Sprint Goal and adjust their plan. At the Sprint Review, a working slideshow would let stakeholders examine navigation and readability directly. The Retrospective would be the place to improve how the team prepared and tested stories. For example, if the no-results test failed to display its message, I would record the steps and actual result, discuss the defects with the developer, and ask the Product Owner to prioritize the correction. The test result would lead to a specific action.

The main benefits of Scrum-Agile for this project were flexibility, early feedback, and clearer connections between requirements and testing. The drawbacks included rework when details were missing, and the effort needed to keep stories and tests current. My missing acceptance criteria were an example of how incomplete preparation could slow later work. For the next Sprint, I would review criteria with the Product Owner and tester before selecting a story, including boundary conditions and empty results. I consider Scrum-Agile the best fit for SNHU Travel because the presentation format and customer focus developed over time. I would

recommend continuing it at ChadaTech while using those lessons to make the next set of stories easier to build, test, and review.